



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Clases, Objetos y Métodos
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	Nivel - Paralelo
Alumnos participantes:	Verdesoto Azogue Kevin Alexander
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. Nombre y

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar un sistema básico de gestión académica en los lenguajes C++ y Java mediante el paradigma de Programación Orientada a Objetos (POO), que permita registrar de forma validada la información y calificaciones de los estudiantes de la asignatura Algoritmos y Lógica de Programación, automatizando el cálculo de sus promedios y la determinación de su estado académico.

Específicos:

- **Diseñar e implementar la clase Estudiante** en ambos lenguajes de programación, aplicando los principios de encapsulamiento mediante el uso de atributos privados, constructores, métodos de acceso (*getters* y *setters*), y funciones específicas para el procesamiento de datos y visualización de la información.
- **Desarrollar mecanismos de validación de datos y lógica automatizada** para garantizar que las calificaciones ingresadas se encuentren estrictamente en el rango de 0 a 10, permitiendo que el sistema calcule el promedio ponderado y defina el estado final (Aprobado o Reprobado) de manera autónoma.
- **Construir una interfaz de consola funcional** que permita el registro masivo de al menos 5 estudiantes y genere reportes estadísticos detallados, mostrando el listado completo de los alumnos y el conteo consolidado de estudiantes aprobados

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos



Listado de equipos y materiales generales empleados en la guía práctica:

- TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:
 - ☐ Plataformas educativas
 - ☐ Simuladores y laboratorios virtuales
 - ☐ Aplicaciones educativas
 - ☐ Recursos audiovisuales
 - ☐ Gamificación
 - ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Problema Propuesto

De acuerdo con la guía SW-AyLP-APE-04: Clases, Objetos y Métodos, la actividad debe ser individual y orientada al desarrollo de programas usando clases, objetos y métodos para reutilizar código y disminuir errores.

Tema del Problema

Sistema básico de control de estudiantes y calificaciones

Enunciado

Desarrollar un programa en C++ y Java que permita registrar información de estudiantes de la asignatura Algoritmos y Lógica de Programación. El sistema debe aplicar programación orientada a objetos mediante clases, objetos y métodos.

- Cada estudiante debe tener:
 - Cédula
 - Nombre
 - Apellido
 - Nota 1
 - Nota 2
 - Nota 3
 - Promedio
 - Estado: Aprobado o Reprobado

Requerimientos del Programa

El estudiante debe crear una clase llamada:

- En C++:

class Estudiante

- En Java:

public class Estudiante

- La clase debe contener:



- Atributos privados
- Constructor
- Métodos get y set
- Método para calcular el promedio
- Método para determinar si aprueba o reprueba
- Método para mostrar la información del estudiante

Funcionalidades mínimas

1. Registrar mínimo 5 estudiantes
2. Ingresar las 3 notas de cada estudiante
3. Calcular automáticamente el promedio
4. Mostrar el listado completo de estudiantes
5. Mostrar cuántos estudiantes aprobaron
6. Mostrar cuántos estudiantes reprobaron
7. Validar que las notas estén entre 0 y 10
8. Comentar correctamente el código

Condición de aprobación

Un estudiante aprueba si su promedio es mayor o igual a 7.00.

Análisis

Vamos a crear un programa ya complejo tanto para C++ como para Java

Algoritmo

- ☐ Inicio del programa.
- ☐ Inicializar el lector de teclado (Scanner) y definir una constante con la cantidad de estudiantes a registrar (fijada en 5).
- ☐ Crear un arreglo o lista física de tamaño 5 para almacenar los objetos de tipo Estudiante.
- ☐ Ciclo de registro: Repetir 5 veces (desde el estudiante 1 hasta el 5):
 - Solicitar y leer la cédula.
 - Solicitar y leer el nombre.
 - Solicitar y leer el apellido.
 - Solicitar la Nota 1. Validar que sea un número y que se encuentre en el rango de 0 a 10. Si no es válida, insistir hasta que lo sea.
 - Solicitar la Nota 2. Validar bajo los mismos criterios (rango de 0 a 10).
 - Solicitar la Nota 3. Validar bajo los mismos criterios (rango de 0 a 10).
 - Instanciar (crear) el objeto Estudiante pasando los datos recolectados al constructor. Internamente, el objeto calcula su propio promedio de forma automática.
 - Almacenar el objeto creado en la posición correspondiente del arreglo.
- ☐ Inicializar dos contadores numéricos en cero: aprobados y reprobados.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



☐ Ciclo de reporte: Recorrer el arreglo de estudiantes desde la primera posición hasta la última:

- Llamar al método del estudiante para imprimir toda su información en pantalla (Cédula, nombre completo, notas, promedio y estado).
- Evaluar el estado del estudiante: Si su estado es igual a "Aprobado", incrementar el contador de aprobados en 1. De lo contrario, incrementar el contador de reprobados en 1.

☐ Imprimir en pantalla el total consolidado de estudiantes aprobados.

☐ Imprimir en pantalla el total consolidado de estudiantes reprobados.

☐ Cerrar el lector de teclado y finalizar el programa.

Diagrama de flujo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026

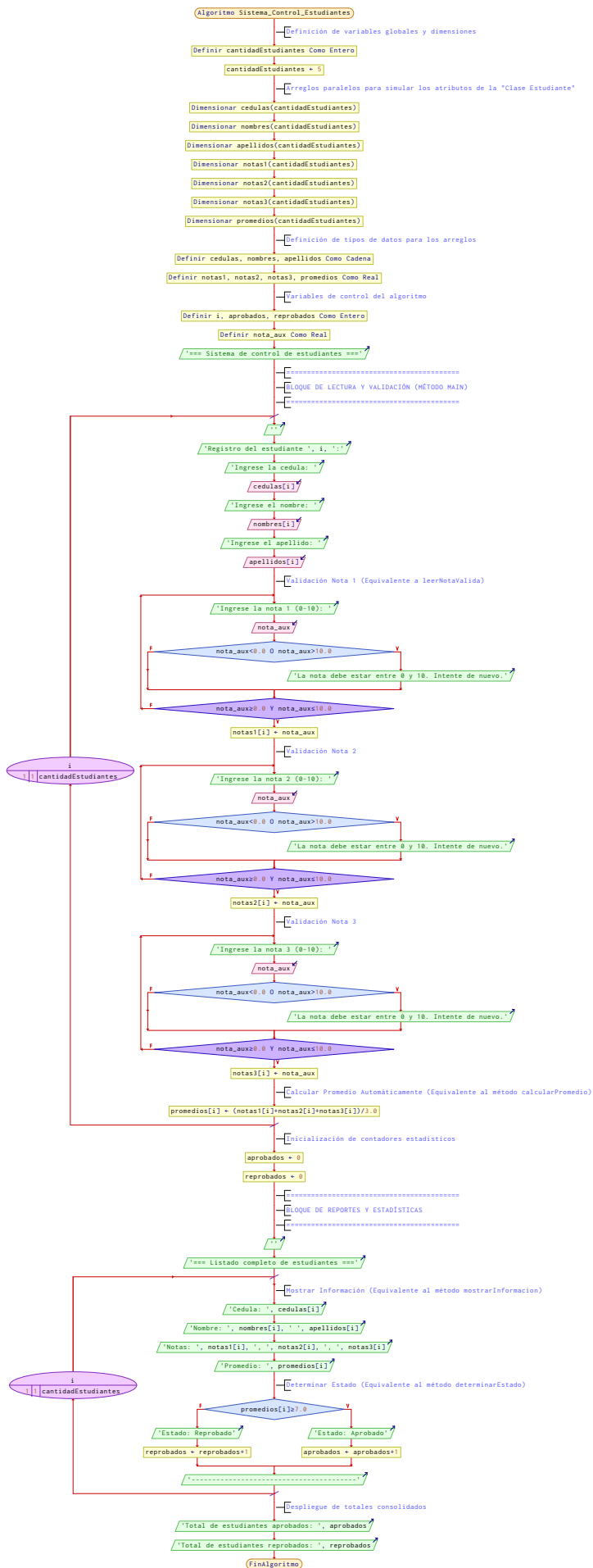




Ilustración 1 Diagrama de flujo

Código en C++

```
#include <iostream>
#include <string>
#include <vector>
#include <limits>
#include <iomanip>

// Clase que representa a un estudiante de Algoritmos y Logica de Programacion
class Estudiante {
private:
    std::string cedula;
    std::string nombre;
    std::string apellido;
    double nota1;
    double nota2;
    double nota3;
    double promedio;

public:
    // Constructor por defecto
    Estudiante()
        : cedula(""), nombre(""), apellido(""), nota1(0.0), nota2(0.0), nota3(0.0), promedio(0.0) {}

    // Constructor con parametros
    Estudiante(const std::string& cedula, const std::string& nombre, const std::string& apellido,
               double nota1, double nota2, double nota3)
        : cedula(cedula), nombre(nombre), apellido(apellido), nota1(nota1), nota2(nota2), nota3(nota3),
        promedio(0.0) {
        calcularPromedio();
    }

    // Metodos get
    std::string getCedula() const { return cedula; }
    std::string getNombre() const { return nombre; }
    std::string getApellido() const { return apellido; }
    double getNota1() const { return nota1; }
    double getNota2() const { return nota2; }
    double getNota3() const { return nota3; }
    double getPromedio() const { return promedio; }

    // Metodos set
    void setCedula(const std::string& value) { cedula = value; }
    void setNombre(const std::string& value) { nombre = value; }
    void setApellido(const std::string& value) { apellido = value; }
```



```
bool setNota1(double value) {
    if (value < 0.0 || value > 10.0) {
        return false;
    }
    nota1 = value;
    return true;
}

bool setNota2(double value) {
    if (value < 0.0 || value > 10.0) {
        return false;
    }
    nota2 = value;
    return true;
}

bool setNota3(double value) {
    if (value < 0.0 || value > 10.0) {
        return false;
    }
    nota3 = value;
    return true;
}

// Calcula el promedio en base a las tres notas
void calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0;
}

// Determina si el estudiante aprueba o reprueba
std::string determinarEstado() const {
    return (promedio >= 7.0) ? "Aprobado" : "Reprobado";
}

// Muestra la informacion completa del estudiante
void mostrarInformacion() const {
    std::cout << "Cedula: " << cedula << "\n";
    std::cout << "Nombre: " << nombre << " " << apellido << "\n";
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Notas: " << nota1 << ", " << nota2 << ", " << nota3 << "\n";
    std::cout << "Promedio: " << promedio << "\n";
    std::cout << "Estado: " << determinarEstado() << "\n";
    std::cout << "-----\n";
}
};

// Lee una nota valida entre 0 y 10
double leerNotaValida(const std::string& mensaje) {
```



```
double nota;
while (true) {
    std::cout << mensaje;
    std::cin >> nota;

    if (std::cin.fail()) {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        std::cout << "Entrada invalida. Ingrese un numero entre 0 y 10.\n";
        continue;
    }

    if (nota < 0.0 || nota > 10.0) {
        std::cout << "La nota debe estar entre 0 y 10. Intente de nuevo.\n";
        continue;
    }

    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    return nota;
}

// Lee una linea de texto desde la entrada estandar
std::string leerLinea(const std::string& mensaje) {
    std::string texto;
    std::cout << mensaje;
    std::getline(std::cin, texto);
    return texto;
}

int main() {
    std::vector<Estudiante> estudiantes;
    const int cantidadEstudiantes = 5;

    std::cout << "=== Sistema de control de estudiantes ===\n";
    for (int i = 0; i < cantidadEstudiantes; ++i) {
        std::cout << "\nRegistro del estudiante " << (i + 1) << ":\n";
        std::string cedula = leerLinea("Ingrese la cedula: ");
        std::string nombre = leerLinea("Ingrese el nombre: ");
        std::string apellido = leerLinea("Ingrese el apellido: ");

        double nota1 = leerNotaValida("Ingrese la nota 1 (0-10): ");
        double nota2 = leerNotaValida("Ingrese la nota 2 (0-10): ");
        double nota3 = leerNotaValida("Ingrese la nota 3 (0-10): ");

        Estudiante estudiante(cedula, nombre, apellido, nota1, nota2, nota3);
        estudiantes.push_back(estudiante);
    }
}
```




```
int aprobados = 0;
int reprobados = 0;

std::cout << "\n=== Listado completo de estudiantes ===\n";
for (const Estudiante& estudiante : estudiantes) {
    estudiante.mostrarInformacion();
    if (estudiante.determinarEstado() == "Aprobado") {
        ++aprobados;
    } else {
        ++reprobados;
    }
}

std::cout << "Total de estudiantes aprobados: " << aprobados << "\n";
std::cout << "Total de estudiantes reprobados: " << reprobados << "\n";

return 0;
}
```

Código en Java

```
import java.util.Scanner;

// Clase publica Estudiante para gestionar datos de estudiantes
public class Estudiante {
    private String cedula;
    private String nombre;
    private String apellido;
    private double nota1;
    private double nota2;
    private double nota3;
    private double promedio;

    // Constructor con parametros
    public Estudiante(String cedula, String nombre, String apellido,
        double nota1, double nota2, double nota3) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.apellido = apellido;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
        calcularPromedio();
    }

    // Metodos get
    public String getCedula() { return cedula; }
```



```
public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
public double getNota1() { return nota1; }
public double getNota2() { return nota2; }
public double getNota3() { return nota3; }
public double getPromedio() { return promedio; }

// Metodos set
public void setCedula(String cedula) { this.cedula = cedula; }
public void setNombre(String nombre) { this.nombre = nombre; }
public void setApellido(String apellido) { this.apellido = apellido; }

public boolean setNota1(double nota1) {
    if (nota1 < 0.0 || nota1 > 10.0) {
        return false;
    }
    this.nota1 = nota1;
    return true;
}

public boolean setNota2(double nota2) {
    if (nota2 < 0.0 || nota2 > 10.0) {
        return false;
    }
    this.nota2 = nota2;
    return true;
}

public boolean setNota3(double nota3) {
    if (nota3 < 0.0 || nota3 > 10.0) {
        return false;
    }
    this.nota3 = nota3;
    return true;
}

// Calcula el promedio de las tres notas
public void calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0;
}

// Devuelve el estado del estudiante segun el promedio
public String determinarEstado() {
    return promedio >= 7.0 ? "Aprobado" : "Reprobado";
}

// Muestra la informacion del estudiante por consola
public void mostrarInformacion() {
```



```
System.out.println("Cedula: " + cedula);
System.out.println("Nombre: " + nombre + " " + apellido);
System.out.printf("Notas: %.2f, %.2f, %.2f%n", nota1, nota2, nota3);
System.out.printf("Promedio: %.2f%n", promedio);
System.out.println("Estado: " + determinarEstado());
System.out.println("-----");
}

private static double leerNotaValida(Scanner scanner, String mensaje) {
    double nota;
    while (true) {
        System.out.print(mensaje);
        if (!scanner.hasNextDouble()) {
            System.out.println("Entrada invalida. Ingrese un numero entre 0 y 10.");
            scanner.next();
            continue;
        }
        nota = scanner.nextDouble();
        scanner.nextLine();
        if (nota < 0.0 || nota > 10.0) {
            System.out.println("La nota debe estar entre 0 y 10. Intente nuevamente.");
            continue;
        }
        return nota;
    }
}

private static String leerTexto(Scanner scanner, String mensaje) {
    System.out.print(mensaje);
    return scanner.nextLine();
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    final int cantidadEstudiantes = 5;
    Estudiante[] estudiantes = new Estudiante[cantidadEstudiantes];

    System.out.println("=== Sistema de control de estudiantes ===");
    for (int i = 0; i < cantidadEstudiantes; i++) {
        System.out.println("\nRegistro del estudiante " + (i + 1) + ":");
        String cedula = leerTexto(scanner, "Ingrese la cedula: ");
        String nombre = leerTexto(scanner, "Ingrese el nombre: ");
        String apellido = leerTexto(scanner, "Ingrese el apellido: ");
        double nota1 = leerNotaValida(scanner, "Ingrese la nota 1 (0-10): ");
        double nota2 = leerNotaValida(scanner, "Ingrese la nota 2 (0-10): ");
        double nota3 = leerNotaValida(scanner, "Ingrese la nota 3 (0-10): ");

        estudiantes[i] = new Estudiante(cedula, nombre, apellido, nota1, nota2, nota3);
    }
}
```



```
}

int aprobados = 0;
int reprobados = 0;

System.out.println("\n=== Listado completo de estudiantes ===");
for (Estudiante estudiante : estudiantes) {
    estudiante.mostrarInformacion();
    if (estudiante.determinarEstado().equals("Aprobado")) {
        aprobados++;
    } else {
        reprobados++;
    }
}

System.out.println("Total de estudiantes aprobados: " + aprobados);
System.out.println("Total de estudiantes reprobados: " + reprobados);
scanner.close();
}
```

Caso de Prueba C++



```
=== Listado completo de estudiantes ===
```

```
Cedula: 1850224187
```

```
Nombre: Kevin Verdesoto
```

```
Notas: 9.00, 9.00, 9.00
```

```
Promedio: 9.00
```

```
Estado: Aprobado
```

```
-----
```

```
Cedula: 1865765465
```

```
Nombre: Brandom Calvopiña
```

```
Notas: 7.00, 6.00, 7.00
```

```
Promedio: 6.67
```

```
Estado: Reprobado
```

```
-----
```

```
Cedula: 1899887656
```

```
Nombre: Bryan Nuñez
```

```
Notas: 5.00, 10.00, 7.00
```

```
Promedio: 7.33
```

```
Estado: Aprobado
```

```
-----
```

```
Cedula: 1877654543
```

```
Nombre: Joan Supe
```

```
Notas: 7.00, 9.00, 6.00
```

```
Promedio: 7.33
```

```
Estado: Aprobado
```

```
-----
```

```
Cedula: 1876554563
```

```
Nombre: Antonella Narvaez
```

```
Notas: 9.00, 10.00, 9.00
```

```
Promedio: 9.33
```

```
Estado: Aprobado
```

```
-----
```

Ilustración 2 Caso de prueba en C++

Caso de Prueba Java



```
Entrada invalida. Ingrese un numero entre 0 y 10.  
Ingrese la nota 3 (0-10): 9.5
```

```
=== Listado completo de estudiantes ===
```

```
Cedula: 1850224187
```

```
Nombre: Kevin Verdesoto
```

```
Notas: 8.00, 9.00, 10.00
```

```
Promedio: 9.00
```

```
Estado: Aprobado
```

```
-----  
Cedula: 1876774563
```

```
Nombre: Bryan Nuñez
```

```
Notas: 9.00, 9.00, 9.00
```

```
Promedio: 9.00
```

```
Estado: Aprobado
```

```
-----  
Cedula: 1876665456
```

```
Nombre: Joan Supe
```

```
Notas: 5.00, 7.00, 8.00
```

```
Promedio: 6.67
```

```
Estado: Reprobado
```

```
-----  
Cedula: 1876554348
```

```
Nombre: Brandom Calvopiña
```

```
Notas: 6.00, 9.00, 10.00
```

```
Promedio: 8.33
```

```
Estado: Aprobado
```

```
-----  
Cedula: 1866706179
```

```
Nombre: Antonella Narvaez
```

```
Notas: 10.00, 9.00, 9.50
```

```
Promedio: 9.50
```

```
Estado: Aprobado
```

```
-----  
Total de estudiantes aprobados: 4
```

```
Total de estudiantes reprobados: 1
```

Ilustración 3 Caso de Prueba en Java

Link del Github

<https://github.com/Kevin-Verdesoto34/APE-04-Clases-Objetos-y-M-todos>

2.7 Resultados obtenidos

Con esto me e dado cuenta que para C++ y Java tienen ciertos cambios en su estructura del código, aunque a simple vista se parezcan por su estructura, con respecto al código obtuve resultados positivos como no usar mucha memoria en la Pc y que este mismo no de error además me di cuenta que esto facilita a los profesores e ingenieros a tener un control más limpio y eficiente de los datos de sus estudiantes.

2.8 Habilidades blandas empleadas en la práctica



- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Cumplimiento de Objetivos Académicos: Se logró modelar con éxito la abstracción del entorno universitario real mediante la Programación Orientada a Objetos en Java, aislando los datos sensibles del estudiante mediante atributos privados y garantizando el acceso controlado por medio de métodos *getters* y *setters*.

Automatización Eficiente de Procesos: La lógica implementada en el constructor para invocar inmediatamente al método `calcularPromedio()` optimiza el rendimiento del software, garantizando que todo estudiante instanciado cuente de forma nativa con sus métricas calculadas sin necesidad de llamadas externas manuales en el programa principal.

Robustez ante Errores de Usuario: El diseño del método auxiliar `leerNotaValida` cumple de forma estricta con el requerimiento de validación (rango 0 a 10), impidiendo activamente el desbordamiento o la corrupción de los datos estadísticos finales del sistema mediante bucles de control repetitivos.

Modularidad y Escalabilidad: Al encapsular el comportamiento del sistema dentro de funciones dedicadas (`determinarEstado`, `mostrarInformacion`), el código fuente resulta altamente legible, limpio y modular, facilitando futuras modificaciones en los criterios de aprobación sin alterar la estructura central del programa.

2.10 Recomendaciones

Desacoplamiento de la Interfaz de Usuario: Se recomienda separar los métodos de utilidad de lectura de consola (`leerNotaValida` y `leerTexto`) de la clase de entidad `Estudiante`. Idealmente, estos métodos deberían pertenecer a la clase que aloja el `main` o a una clase controladora independiente, manteniendo la pureza y la única responsabilidad del objeto de datos.

Validación de Formatos de Cédula y Texto: El programa actual acepta cualquier cadena de caracteres para la cédula, nombres y apellidos. Se sugiere implementar expresiones regulares (`Regex`) para asegurar que el documento de identidad contenga únicamente caracteres numéricos y que los nombres no almacenen símbolos especiales o números vacíos.

Migración a Colecciones Dinámicas: Reemplazar el arreglo estático de tamaño fijo (`Estudiante[]`) por una lista dinámica como `ArrayList<Estudiante>`. Esto eliminará la limitación estricta de registrar únicamente 5 usuarios, dotando al sistema de la capacidad de adaptarse a un volumen variable de alumnos en tiempo de ejecución.

Implementación de Persistencia de Datos: El software actual pierde toda la información registrada al cerrarse la consola de comandos. Se recomienda integrar el manejo de flujos de archivos (`FileWriter` / `FileReader` en Java) para almacenar de forma permanente el registro en un documento de texto plano (`.txt`), permitiendo recuperar la información en ejecuciones posteriores.

2.11 Referencias bibliográficas

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

2.12 Anexos

Código en C++

```
#include <iostream>
#include <string>
```



```
#include <vector>
#include <limits>
#include <iomanip>

// Clase que representa a un estudiante de Algoritmos y Logica de
// Programacion
class Estudiante {
private:
    std::string cedula;
    std::string nombre;
    std::string apellido;
    double nota1;
    double nota2;
    double nota3;
    double promedio;

public:
    // Constructor por defecto
    Estudiante()
        : cedula(""), nombre(""), apellido(""), nota1(0.0), nota2(0.0),
        nota3(0.0), promedio(0.0) {}

    // Constructor con parametros
    Estudiante(const std::string& cedula, const std::string& nombre,
    const std::string& apellido,
        double nota1, double nota2, double nota3)
        : cedula(cedula), nombre(nombre), apellido(apellido),
        nota1(nota1), nota2(nota2), nota3(nota3), promedio(0.0) {
        calcularPromedio();
    }

    // Metodos get
    std::string getCedula() const { return cedula; }
    std::string getNombre() const { return nombre; }
    std::string getApellido() const { return apellido; }
    double getNota1() const { return nota1; }
    double getNota2() const { return nota2; }
    double getNota3() const { return nota3; }
    double getPromedio() const { return promedio; }

    // Metodos set
    void setCedula(const std::string& value) { cedula = value; }
    void setNombre(const std::string& value) { nombre = value; }
    void setApellido(const std::string& value) { apellido = value; }

    bool setNota1(double value) {
        if (value < 0.0 || value > 10.0) {
            return false;
        }
    }
}
```




```
}
    nota1 = value;
    return true;
}

bool setNota2(double value) {
    if (value < 0.0 || value > 10.0) {
        return false;
    }
    nota2 = value;
    return true;
}

bool setNota3(double value) {
    if (value < 0.0 || value > 10.0) {
        return false;
    }
    nota3 = value;
    return true;
}

// Calcula el promedio en base a las tres notas
void calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0;
}

// Determina si el estudiante aprueba o reprueba
std::string determinarEstado() const {
    return (promedio >= 7.0) ? "Aprobado" : "Reprobado";
}

// Muestra la informacion completa del estudiante
void mostrarInformacion() const {
    std::cout << "Cedula: " << cedula << "\n";
    std::cout << "Nombre: " << nombre << " " << apellido << "\n";
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Notas: " << nota1 << ", " << nota2 << ", " << nota3
<< "\n";
    std::cout << "Promedio: " << promedio << "\n";
    std::cout << "Estado: " << determinarEstado() << "\n";
    std::cout << "-----\n";
}
};

// Lee una nota valida entre 0 y 10
double leerNotaValida(const std::string& mensaje) {
    double nota;
    while (true) {
```



```
std::cout << mensaje;
std::cin >> nota;

if (std::cin.fail()) {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    std::cout << "Entrada invalida. Ingrese un numero entre 0 y
10.\n";
    continue;
}

if (nota < 0.0 || nota > 10.0) {
    std::cout << "La nota debe estar entre 0 y 10. Intente de
nuevo.\n";
    continue;
}

std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
return nota;
}

// Lee una linea de texto desde la entrada estandar
std::string leerLinea(const std::string& mensaje) {
    std::string texto;
    std::cout << mensaje;
    std::getline(std::cin, texto);
    return texto;
}

int main() {
    std::vector<Estudiante> estudiantes;
    const int cantidadEstudiantes = 5;

    std::cout << "=== Sistema de control de estudiantes ===\n";
    for (int i = 0; i < cantidadEstudiantes; ++i) {
        std::cout << "\nRegistro del estudiante " << (i + 1) << ":\n";
        std::string cedula = leerLinea("Ingrese la cedula: ");
        std::string nombre = leerLinea("Ingrese el nombre: ");
        std::string apellido = leerLinea("Ingrese el apellido: ");

        double nota1 = leerNotaValida("Ingrese la nota 1 (0-10): ");
        double nota2 = leerNotaValida("Ingrese la nota 2 (0-10): ");
        double nota3 = leerNotaValida("Ingrese la nota 3 (0-10): ");
```



```
Estudiante estudiante(cedula, nombre, apellido, nota1, nota2,
nota3);
    estudiantes.push_back(estudiante);
}

int aprobados = 0;
int reprobados = 0;

std::cout << "\n=== Listado completo de estudiantes ===\n";
for (const Estudiante& estudiante : estudiantes) {
    estudiante.mostrarInformacion();
    if (estudiante.determinarEstado() == "Aprobado") {
        ++aprobados;
    } else {
        ++reprobados;
    }
}

std::cout << "Total de estudiantes aprobados: " << aprobados << "\n";
std::cout << "Total de estudiantes reprobados: " << reprobados <<
"\n";

return 0;
}
```

Caso de prueba en C++



=== Listado completo de estudiantes ===

Cedula: 1850224187

Nombre: Kevin Verdesoto

Notas: 9.00, 9.00, 9.00

Promedio: 9.00

Estado: Aprobado

Cedula: 1865765465

Nombre: Brandom Calvopiña

Notas: 7.00, 6.00, 7.00

Promedio: 6.67

Estado: Reprobado

Cedula: 1899887656

Nombre: Bryan Nuñez

Notas: 5.00, 10.00, 7.00

Promedio: 7.33

Estado: Aprobado

Cedula: 1877654543

Nombre: Joan Supe

Notas: 7.00, 9.00, 6.00

Promedio: 7.33

Estado: Aprobado

Cedula: 1876554563

Nombre: Antonella Narvaez

Notas: 9.00, 10.00, 9.00

Promedio: 9.33

Estado: Aprobado

Ilustración 4 Caso de Prueba en C++

Código en Java

```
import java.util.Scanner;

// Clase publica Estudiante para gestionar datos de estudiantes
public class Estudiante {
    private String cedula;
    private String nombre;
    private String apellido;
    private double nota1;
    private double nota2;
    private double nota3;
    private double promedio;

    // Constructor con parametros
    public Estudiante(String cedula, String nombre, String apellido,
                      double nota1, double nota2, double nota3) {
        this.cedula = cedula;
        this.nombre = nombre;
```



```
this.apellido = apellido;
this.nota1 = nota1;
this.nota2 = nota2;
this.nota3 = nota3;
calcularPromedio();
}

// Metodos get
public String getCedula() { return cedula; }
public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
public double getNota1() { return nota1; }
public double getNota2() { return nota2; }
public double getNota3() { return nota3; }
public double getPromedio() { return promedio; }

// Metodos set
public void setCedula(String cedula) { this.cedula = cedula; }
public void setNombre(String nombre) { this.nombre = nombre; }
public void setApellido(String apellido) { this.apellido = apellido; }
}

public boolean setNota1(double nota1) {
    if (nota1 < 0.0 || nota1 > 10.0) {
        return false;
    }
    this.nota1 = nota1;
    return true;
}

public boolean setNota2(double nota2) {
    if (nota2 < 0.0 || nota2 > 10.0) {
        return false;
    }
    this.nota2 = nota2;
    return true;
}

public boolean setNota3(double nota3) {
    if (nota3 < 0.0 || nota3 > 10.0) {
        return false;
    }
    this.nota3 = nota3;
    return true;
}

// Calcula el promedio de las tres notas
public void calcularPromedio() {
```



```
        promedio = (nota1 + nota2 + nota3) / 3.0;
    }

    // Devuelve el estado del estudiante segun el promedio
    public String determinarEstado() {
        return promedio >= 7.0 ? "Aprobado" : "Reprobado";
    }

    // Muestra la informacion del estudiante por consola
    public void mostrarInformacion() {
        System.out.println("Cedula: " + cedula);
        System.out.println("Nombre: " + nombre + " " + apellido);
        System.out.printf("Notas: %.2f, %.2f, %.2f\n", nota1, nota2,
nota3);
        System.out.printf("Promedio: %.2f\n", promedio);
        System.out.println("Estado: " + determinarEstado());
        System.out.println("-----");
    }

    private static double leerNotaValida(Scanner scanner, String mensaje)
    {
        double nota;
        while (true) {
            System.out.print(mensaje);
            if (!scanner.hasNextDouble()) {
                System.out.println("Entrada invalida. Ingrese un numero
entre 0 y 10.");
                scanner.next();
                continue;
            }
            nota = scanner.nextDouble();
            scanner.nextLine();
            if (nota < 0.0 || nota > 10.0) {
                System.out.println("La nota debe estar entre 0 y 10.
Intente nuevamente.");
                continue;
            }
            return nota;
        }
    }

    private static String leerTexto(Scanner scanner, String mensaje) {
        System.out.print(mensaje);
        return scanner.nextLine();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```



```
final int cantidadEstudiantes = 5;
Estudiante[] estudiantes = new Estudiante[cantidadEstudiantes];

System.out.println("=== Sistema de control de estudiantes ===");
for (int i = 0; i < cantidadEstudiantes; i++) {
    System.out.println("\nRegistro del estudiante " + (i + 1) +
":");
    String cedula = leerTexto(scanner, "Ingrese la cedula: ");
    String nombre = leerTexto(scanner, "Ingrese el nombre: ");
    String apellido = leerTexto(scanner, "Ingrese el apellido:
");
    double nota1 = leerNotaValida(scanner, "Ingrese la nota 1 (0-
10): ");
    double nota2 = leerNotaValida(scanner, "Ingrese la nota 2 (0-
10): ");
    double nota3 = leerNotaValida(scanner, "Ingrese la nota 3 (0-
10): ");

    estudiantes[i] = new Estudiante(cedula, nombre, apellido,
nota1, nota2, nota3);
}

int aprobados = 0;
int reprobados = 0;

System.out.println("\n=== Listado completo de estudiantes ===");
for (Estudiante estudiante : estudiantes) {
    estudiante.mostrarInformacion();
    if (estudiante.determinarEstado().equals("Aprobado")) {
        aprobados++;
    } else {
        reprobados++;
    }
}

System.out.println("Total de estudiantes aprobados: " +
aprobados);
System.out.println("Total de estudiantes reprobados: " +
reprobados);
scanner.close();
}
```

Caso de Prueba en Java



```
Entrada invalida. Ingrese un numero entre 0 y 10.  
Ingrese la nota 3 (0-10): 9.5
```

```
=== Listado completo de estudiantes ===
```

```
Cedula: 1850224187  
Nombre: Kevin Verdesoto  
Notas: 8.00, 9.00, 10.00  
Promedio: 9.00  
Estado: Aprobado
```

```
-----  
Cedula: 1876774563  
Nombre: Bryan Nuñez  
Notas: 9.00, 9.00, 9.00  
Promedio: 9.00  
Estado: Aprobado
```

```
-----  
Cedula: 1876665456  
Nombre: Joan Supe  
Notas: 5.00, 7.00, 8.00  
Promedio: 6.67  
Estado: Reprobado
```

```
-----  
Cedula: 1876554348  
Nombre: Brandom Calvopiña  
Notas: 6.00, 9.00, 10.00  
Promedio: 8.33  
Estado: Aprobado
```

```
-----  
Cedula: 1866706179  
Nombre: Antonella Narvaez  
Notas: 10.00, 9.00, 9.50  
Promedio: 9.50  
Estado: Aprobado
```

```
-----  
Total de estudiantes aprobados: 4  
Total de estudiantes reprobados: 1
```

Ilustración 5 Caso de Prueba Java

Link del Github

<https://github.com/Kevin-Verdesoto34/APE-04-Clases-Objetos-y-M-todos>



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Ilustración 1 Diagrama de flujo	6
Ilustración 2 Caso de prueba en C++	13
Ilustración 3 Caso de Prueba en Java	14
Ilustración 4 Caso de Prueba en C++	20
Ilustración 5 Caso de Prueba Java	24